

Simulating a Particle Detector and Particle Decay in C++

Aidan Hughes

11306492

School of Physics and Astronomy

University of Manchester

Third Year Code Report

May 2026

Abstract

The aim of this project was to utilize object oriented programming practices to create a particle detector and particle decay simulators in C++. It consists of two main inheritance chains, one for the particle classes and another for sub-detector classes. These classes, and their respective container classes ParticleBunch and Detector, practice encapsulation by storing data members as private or protected with access via public member functions. The use of virtual and pure virtual functions in the abstract parent classes of the hierarchy enables polymorphism in the child classes.

1 Introduction

The aim of this project was to create a C++ library that simulates a particle detector and its detecting functionality. Particle detectors compose of multiple sub-detector types, each of which measure different types of particles. The tracker, the first sub-detector a particle will pass through, uses magnetic fields to measure particle paths, so can only measure charged particles. Next are the calorimeters, which are split into two, an electromagnetic and a hadronic calorimeter. The electromagnetic calorimeter causes electrons and photons to trigger a particle shower which is measured to find their energy. The hadronic calorimeter does the same but for protons and neutrons. Finally, due to muons high energy, they are not detected by the calorimeters, and p This was done through two inheritance chains, one for the sub-detectors with the base class SubDetector and one for particles with base class Particle.

2 Code Implementation

Object oriented programming (OOP) is built on four pillars: encapsulation, inheritance, polymorphism and abstraction.

PLAN OOP EXAMPLES encapsulation - all classes, particle probaly best inheritance - particle hierarchy polymorphism static - overloaded function - constuctors dynamic - info() function on particles - detector flags abstraction - particle type enum class

FEATURES USED enum class for particle type - validation in one place - easier to scale switch cases than if enables - re used for Neutrino flavour deque to do particle decay unordered map to do key value pairs rand() for efficiency / misdetection calulation

3 Results

```
1      int main()
2      {
3          return 0;
4      }
```

4 Discussion

errors on detected value. Extra abstraction in detect function.